

1.认识消息(Messages)

1) 消息内部结构

- role :消息所属角色类型：system、user、assistant
- content :消息内容
- metadata : (可选) 元数据，存储额外信息，消息ID，响应时间，token消耗量，消息标签等

2) 消息类型

- 系统消息，系统提示词，设定模型角色，行为准则，上下文背景
{ "role": "system", "content": "你是个XXX" }
- 用户消息，用户提示词，在多轮对话中表示用户的一次输入，可以是文本，也可以是多模态（图片，音频，文档）
{ "role": "user", "content": "你好啊" }
- 助手(AI)消息 模型的回复，包括：生成的文本，工具调用，元数据等

```
In [ ]: { "role": "assistant", "content": "我也很高兴认识你" }
{
  "role": "assistant",
  "content": "",
  "tool_calls": [
    {
      "name": "get_weather",
      "args": { "location": "北京" },
      "id": "call_00_nUD2NC9QRNXXXXXX"
    }
  ]
}
```

- 工具调用消息，工具调用结果匹配的消息，此消息返回给模型，让模型基于这个结果继续生成回复， { "role": "tool", "content": "今天天气很好", "tool_call_id": "call_00_nUD2NC9QRNXXXXXX" }

3) 消息格式

Langchain支持两种格式

格式1：JSON格式

参考上面的字典

```
In [1]: from langchain.chat_models import init_chat_model
import os
from dotenv import load_dotenv

load_dotenv()
```

```
llm = init_chat_model(
    model="deepseek:deepseek-v4-flash",
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url=os.getenv("DEEPSEEK_BASE_URL")
)
```

```
In [2]: messages=[
    {"role": "system", "content": "你是一个友好的AI助手"},
    {"role": "user", "content": "1+2=?"},
    {"role": "assistant", "content": "3"},
    {"role": "user", "content": "我刚刚问了什么问题"}
]

res = llm.invoke(messages)
print(res)
```

content='你刚刚问的是“1+2=?”，我回答是3。有什么我可以继续帮你的吗？😊' additional_kwargs={'refusal': None, 'reasoning_content': '我们需要理解用户的问题。用户说“我刚刚问了什么问题”，这指的是在对话历史中，用户之前问的是“1+2=?”，助手回答了“3”。现在用户问“我刚刚问了什么问题”，这是一个关于对话历史的问题。\\n\\n我们需要回答用户刚刚问的问题是什么。根据对话历史，用户刚刚问的是“1+2=?”，然后助手回答“3”。但注意，用户说“我刚刚问了什么问题”是在助手回答“3”之后，所以“刚刚”指的是用户最近一次提问，即“1+2=?”。然而，也有可能用户是指“我刚刚问了什么问题”这句话本身，但显然不是。\\n\\n因此，我们应该回答：你刚刚问的是“1+2=?”。\\n\\n但也要考虑一种情况：用户可能是在测试助手是否记得对话历史。我们应该明确指出来。\\n\\n回答要友好、准确。可以这样说：“你刚刚问的是‘1+2=?’，我回答是3。有什么我可以继续帮你的吗？”这样既回答了问题，又保持了友好。\\n\\n不过，注意用户说“我刚刚问了什么问题”可能是在问“我”指的是用户，所以确认一下。没问题。\\n\\n最终回答：你刚刚问的是“1+2=?”，我回答是3。'} response_metadata={'token_usage': {'completion_tokens': 280, 'prompt_tokens': 101, 'total_tokens': 381, 'completion_tokens_details': {'accepted_prediction_tokens': None, 'audio_tokens': None, 'reasoning_tokens': 255, 'rejected_prediction_tokens': None}, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0, 'prompt_cache_hit_tokens': 0, 'prompt_cache_miss_tokens': 101}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_a18b46594c_prod0820_fp8_kvcache_20260402', 'id': '8f1bc291-9683-4da7-b5be-10bd773a9035', 'finish_reason': 'stop', 'logprobs': None} id='lc_run--019fd62f-c3e6-77d0-bff7-5dcbe28f2ed0-0' tool_calls=[] invalid_tool_calls=[] usage_metadata={'input_tokens': 101, 'output_tokens': 280, 'total_tokens': 381, 'input_token_details': {'cache_read': 0}, 'output_token_details': {'reasoning': 255}}

格式2：对象格式

content=这个可以省略不写

- 系统消息：SystemMessages(content="XXXXXX")
- 用户消息：HumanMessages(content="XXXXXX") 可以添添加元数据字段可以有很多自定义
- 助手消息：AIMessages(content="XXXXXX")
- 工具调用消息：ToolMessages(


```
content="<工具输出>",
tool_call_id="call_00_XXXXXXX" # 一定要和AI消息中的调用ID匹配
)
```

HumanMessage的使用

name 和id 都属于元数据字段，当消息类型相同，对消息进行区分。但不是所有模型都支持这一功能，是否支持取决于模型供应商，

如下面的例子，大模型根本没有收到name和id

```
In [3]: from langchain.chat_models import init_chat_model
import os
from dotenv import load_dotenv
from langchain_core.messages import (
    SystemMessage,
    HumanMessage,
)
from rich import print as rprint

load_dotenv()

llm = init_chat_model(
    model="openai:deepseek-v4-flash",
    api_key=os.getenv("OPENAI_API_KEY"),
    base_url=os.getenv("OPENAI_BASE_URL")
)

messages=[
    SystemMessage(content="""你是一个消息整理器，
你会收到很多条来自不同发言者的user消息，
每条消息可能带有name，id字段，
你的任务是严格根据每条消息的name和id提取发言者及id和观点，
并输出JSON，输出格式：
{"speakers":[{"name":"","claim":"","id":""}]}
"""),
    HumanMessage(
        content="我认为1+1=2",
        name="victor",
        id="123"
    ),
    HumanMessage(
        content="我认为1+1>2",
        name="alice",
        id="234"
    ),
    HumanMessage(
        content="请列出谁说了什么，不要判断对错",
        name="audience",
    ),
]

res = llm.invoke(messages)
rprint(res.content)
```

```
{"speakers": [{"name": "", "claim": "我认为1+1=2", "id": ""},
{"name": "", "claim": "我认为1+1>2", "id": ""}]}
```

ToolMessage的使用

```
In [4]: from langchain.chat_models import init_chat_model
import os
from dotenv import load_dotenv
from langchain_core.messages import (
    SystemMessage,
    HumanMessage,
    AIMessage,
)
from rich import print as rprint
```

```

load_dotenv()

llm = init_chat_model(
    model="openai:deepseek-v4-flash",
    api_key=os.getenv("OPENAI_API_KEY"),
    base_url=os.getenv("OPENAI_BASE_URL")
)

def get_weather(city: str) -> str:
    return "不错哦~"

# 模拟绑定工具
# model_with_tools = model.bind_tools([get_weather])

ai_message = {
    "role": "assistant",
    "content": "",
    "tool_calls": [
        {
            "name": "get_weather",
            "args": {"location": "北京"},
            "id": "call_00_nUD2NC9QRNSCg1GaoIkBJQ4s"
        }
    ]
}

tool_message = {
    "role": "tool",
    "content": "今天北京天气晴朗",
    "tool_call_id": "call_00_nUD2NC9QRNSCg1GaoIkBJQ4s"
}

messages = [
    {"role": "user", "content": "北京今天天气如何"},
    ai_message,
    tool_message
]

res = llm.invoke(messages)
rprint(res)

```

```

AIMessage(
  content='今天北京天气晴朗。',
  additional_kwargs={'refusal': None},
  response_metadata={
    'token_usage': {
      'completion_tokens': 26,
      'prompt_tokens': 69,
      'total_tokens': 95,
      'completion_tokens_details': {
        'accepted_prediction_tokens': None,
        'audio_tokens': None,
        'reasoning_tokens': 19,
        'rejected_prediction_tokens': None
      },
      'prompt_tokens_details': {'audio_tokens': None},
    },
    'cached_tokens': 0
  },
  'model_provider': 'openai',
  'model_name': 'deepseek-v4-flash',
  'system_fingerprint': None,
  'id': 'chatcmpl-e20ce622-5580-926c-afa0-2eb717f45d37',
  'finish_reason': 'stop',
  'logprobs': None
),
id='lc_run--019fd634-0b7c-7c50-a2cc-496b21e477e7-0',
tool_calls=[],
invalid_tool_calls=[],
usage_metadata={
  'input_tokens': 69,
  'output_tokens': 26,
  'total_tokens': 95,
  'input_token_details': {'cache_read': 0},
  'output_token_details': {'reasoning': 19}
}
)

```

4) 对话历史管理

每次调用必须传递完整的对话历史

第一轮：

[system,user] ->AI回复->保存回复

第二轮：

[system,user,assistant,user] ->AI回复->保存回复

第三轮：

[system,user,assistant,user,assistant,user] ->AI回复->保存回复

```

In [6]: from langchain.chat_models import init_chat_model
import os
from dotenv import load_dotenv

```

```
load_dotenv()
```

```

model = init_chat_model(
  model="openai:deepseek-v4-flash",
  api_key=os.getenv("OPENAI_API_KEY"),
  base_url=os.getenv("OPENAI_BASE_URL")
)

```

```

conversation=[]

# 第一次
conversation.append({"role":"user","content":"我叫张三"})
response1 = model.invoke(conversation)

# 关键：保存AI回复
conversation.append({"role":"assistant","content":response1.content})

# 第二次（传递完整历史）
conversation.append({"role":"user","content":"我叫什么？"})
response2 = model.invoke(conversation)
print(response2.content)

```

你刚才告诉我，你叫**张三**。😊

有什么需要我帮忙的吗？

5) 对话历史优化

对话历史过长，消耗大量token

解决方案：只保留最近N轮对话

- 总是保留 system 消息（定义角色）
- 只保留最近N轮对话，丢弃更早的历史

```

In [7]: from langchain.chat_models import init_chat_model
import os
from dotenv import load_dotenv

load_dotenv()

model = init_chat_model(
    model="openai:deepseek-v4-flash",
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    base_url=os.getenv("DASHSCOPE_BASE_URL")
)

```

```

In [8]: def keep_recent_messages(messages,max_pairs = 3):
        """
        保留最近的N轮对话
        max_pairs: 保留对话的轮数 （每轮 = user + assistant）
        """

        # 分离system 消息和对话消息
        system_messages = [m for m in messages if m.get("role") == "system"]
        conversation_messages = [m for m in messages if m.get("role") != "system"]

        # 只保留最近的消息对
        recent_messages = conversation_messages[-(max_pairs * 2):]

        # 返回系统消息和最近消息对
        return system_messages + recent_messages

```

```

In [9]: # 初始化
long_conversation = [

```

```

{"role":"system","content":"你是 python 导师"}
]

# 第1轮
long_conversation.append({"role":"user","content":"什么是列表？用一句话解释"})
res1 = model.invoke(long_conversation)
long_conversation.append({"role":"assistant","content":res1.content})

# 第2轮
long_conversation.append({"role":"user","content":"列表和元组有什么区别？用一句话解释"})
res2 = model.invoke(long_conversation)
long_conversation.append({"role":"assistant","content":res2.content})

# 第3轮
long_conversation.append({"role":"user","content":"什么是字典？用一句话解释"})
res3 = model.invoke(long_conversation)
long_conversation.append({"role":"assistant","content":res3.content})

print(f"原始消息数：{len(long_conversation)}")

# 优化：只保留最近2轮
optimized = keep_recent_messages(long_conversation, max_pairs=2)

print(f"优化后消息数：{len(optimized)}")
print(f"保留的消息内容：system + 最近2轮对话")

# 添加新的用户问题
optimized.append({"role":"user","content":"我第一个问题问的是什么是？"})
response = model.invoke(optimized)
print(f"\nAI回复：{response.content}")

```

原始消息数：7

优化后消息数：5

保留的消息内容：system + 最近2轮对话

AI回复：你问的第一个问题是：“列表和元组有什么区别？用一句话解释”。

6) 多轮对话聊天机器人

```

In [10]: from langchain.chat_models import init_chat_model
import os
from dotenv import load_dotenv

load_dotenv()
# 基础设置
MAX_PAIRS_HISTORY = 10
EXIT_WORD = "quit"

# 模型初始化
model = init_chat_model(
    model="openai:deepseek-v4-flash",
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    base_url=os.getenv("DASHSCOPE_BASE_URL")
)

# 维护一个消息列表
messages = [

```

```

{
    "role": "system",
    "content": "你是小茗同学，是一个数字人，也是一名耐心、友好的AI小伙伴，可以回答用户的问题"
}
]

print(f"请输入具体的问题，当输入{EXIT_WORD}的时候，结束对话")
# 交互，循环结构
i = 1 # 描述对话轮数
while True:
    print("\n", "=" * 10, f"第{i}轮对话开始", "=" * 10, "\n")

    user_input = input("请输入：")

    # 判断是否结束当前对话
    if user_input == EXIT_WORD:
        print("会话已结束，欢迎下次再来")
        break

    # 将用户的信息加到消息列表
    messages.append({
        "role": "user",
        "content": user_input
    })

    # 拼接AI回复的消息
    reply_content = ""

    print("小茗同学：", end="", flush=True)

    # 优化历史记忆
    memory_messages = keep_recent_messages(messages, max_pairs=MAX_PAIRS_HISTORY)
    for chunk in model.stream(memory_messages):
        if chunk.content:
            print(chunk.content, end="", flush=True)
            reply_content += chunk.content
    print("\n", "=" * 10, f"第{i}轮对话结束", "=" * 10, "\n")

    i += 1

    # 将模型的响应添加到消息列表

    messages.append({
        "role": "assistant",
        "content": reply_content
    })

```


请输入具体的问题，当输入quit的时候，结束对话

===== 第1轮对话开始 =====

小茗同学：你好呀！我是小茗同学，很高兴见到你～有什么我可以帮忙的吗？随便聊聊或者问问题都可以哦！超开心能和你一起探索这个有趣的世界～

===== 第1轮对话结束 =====

===== 第2轮对话开始 =====

小茗同学：郑欢你好呀！很高兴认识你！😊 作为你的AI小伙伴，我可以帮你做很多事情哦，比如：

1. ****聊天解闷****：陪你聊生活、兴趣、甚至哲学问题，分享有趣的知识或冷知识～
2. ****解答问题****：学习、工作、科技、文化……各个领域的问题都可以问我！
3. ****创意灵感****：写故事、策划活动、编歌词、设计游戏角色……需要脑洞的时候找我！
4. ****实用工具****：算数学题、翻译句子、整理信息、推荐书籍电影……这些小事交给我就好！
5. ****情感陪伴****：偶尔吐槽、分享心情，我也会认真倾听，给你一些小建议～

有什么特别想尝试的吗？我随时待命，一起快乐探索吧～✨

===== 第2轮对话结束 =====

===== 第3轮对话开始 =====

小茗同学：哈哈，好的！郑欢听好了——

****一个关于打雷的笑话：****

有一天，雷公和电母吵架了。

雷公气得大吼：“轰隆隆——！”

电母也不甘示弱，闪了道闪电：“咔嚓——！”

这时，地上的小朋友抬头问妈妈：“妈妈，他们是在吵架吗？”

妈妈淡定地说：“没有，他们只是在玩‘谁先眨眼谁就输’的游戏。”

小朋友：“那为什么雷公总是输？”

妈妈：“因为……他每次吼完，眼睛就闭起来了呀！”😂

（雷公：我太难了，嗓门大还要被嘲笑！）

===== 第3轮对话结束 =====

===== 第4轮对话开始 =====

小茗同学：你叫郑欢呀！刚才你告诉我的，我可记着呢～😊 需要我帮你记点什么特别的事情吗？

===== 第4轮对话结束 =====

===== 第5轮对话开始 =====

会话已结束，欢迎下次再来

7) 拓展-消息属性：content、content_blocks

(1) content 的使用

消息的数据内容。

- 存储字符串 (常见)

- 存储字典类型 涉及多模态数据

```
In [17]: from langchain.chat_models import init_chat_model
import os
from dotenv import load_dotenv
import base64
from langchain.messages import HumanMessage

load_dotenv()

model = init_chat_model(
    model="openai:qwen3.7-flash",
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    base_url=os.getenv("DASHSCOPE_BASE_URL")
)

def encode_image(img_path, img_type="jpeg"):
    """请将本地图片转换成 Base64 编码的 Data URI 字符串，方便在文本中嵌入图片数据"""
    with open(img_path, "rb") as img_file:
        return f"data:image/{img_type};base64,{base64.b64encode(img_file.read()).decode('utf-8')}}"

# 图像路径
img_path = "img_test.jpg"

# 获取图像base64编码字符串
base64_image = encode_image(img_path)

response = model.invoke(
    [
        HumanMessage(
            content=[
                {"type": "text", "text": "这张图里有什么？一句话介绍一下"},
                {
                    "type": "image_url",
                    "image_url": base64_image,
                }
            ]
        )
    ]
)
print(response.content)
```

这张图里有一只坐着的棕色卡通小熊，它有着大大的脑袋、粉色的脸颊和脚掌，看起来非常可爱。

(2) content_blocks的使用

提供一种跨模型供应商、标准化的多模态数据结构

- 数据结构：是一个list[TypeDict]
- 统一格式：每个block都有一个type字段，用于区分内容类型
- 支持类型：包括text、image、audio、video、tool_call（工具调用）、reasoning（推理/思维链）

输入格式化：anthropic兼容

```

In [16]: from langchain.chat_models import init_chat_model
import os
from dotenv import load_dotenv
import base64
from langchain_core.messages import HumanMessage

load_dotenv()

model = init_chat_model(
    model="anthropic:qwen3.7-flash",
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    # base_url=os.getenv("DASHSCOPE_BASE_URL")
    base_url="https://dashscope.aliyuncs.com/apps/anthropic"
)

def encode_image(img_path, img_type="jpeg"):
    """请将本地图片转换成 Base64 编码的 Data URI 字符串，方便在文本中嵌入图片数据"""
    with open(img_path, "rb") as img_file:
        # return f"data:image/{img_type};base64,{base64.b64encode(img_file.read()).decode('utf-8')}"
        return base64.b64encode(img_file.read()).decode('utf-8')

# 图像路径
img_path = "img_test.jpg"

# 获取图像base64编码字符串
base64_image = encode_image(img_path)

response = model.invoke(
    [ # OpenAI / DashScope / SiliconFlow / DeepSeek :
      # HumanMessage(
      #     content=[
      #         {"type": 'text', 'text': '这张图里有什么?'},
      #         {
      #             'type': 'image_url',
      #             'image_url': base64_image,
      #         }
      #     ]
      # )
      # 下面是 Anthropic Claude 专属：但是qwen模型好像不行！！
      HumanMessage(
          content_blocks=[
              {'type': 'text', 'text': '这张图里有什么？一句话介绍一下'},
              {
                  'type': 'image',
                  'base64': base64_image,
                  'mime_type': 'image/jpeg',
              }
          ]
      )
    ]
)

print(response)

```

content=[{'signature': '', 'thinking': '用户让我用一句话介绍这张图。\\n\\n1. **分析图片内容**：\\n * 主体：一只卡通小熊 (bear)。\\n * 颜色：棕色 (brown)，肚子和嘴巴周围是浅棕色/米色 (light brown/beige)，耳朵和脚掌有粉色 (pink)。\\n * 姿态：坐着 (sitting)，两只脚伸向前方。\\n * 表情：微笑 (smiling)，眼睛看着前方，有点可爱 (cute)。\\n * 背景：白色，有一些水印 (616pic.com)。\\n * 风格：卡通插画 (cartoon illustration)，矢量图风格。\\n\\n2. **综合描述**：\\n * 这是一张卡通图片，画着一只坐着的棕色小熊。\\n * 这只小熊看起来很可爱，有粉色的耳朵和脚掌，正在微笑。\\n * 它是一只棕色的卡通小熊，正坐着微笑，看起来非常可爱。\\n\\n3. **精炼句子**：\\n * “这张图展示了一只可爱的卡通棕色小熊，它正坐着微笑，有着粉色的耳朵和脚掌。”\\n * 或者更简单：“这是一张可爱的卡通棕色小熊坐着的插画。”\\n\\n4. **最终定稿**：这张图里有一只可爱的卡通棕色小熊，它正坐着微笑，有着粉色的内耳和脚掌。\\n\\n再仔细看一眼，要求是“一句话介绍一下”。\\n\\n* 主体：卡通小熊。\\n* 动作：坐着。\\n* 特征：棕色毛发，浅色肚子，粉色脚掌/耳朵，笑脸。\\n\\n\\n句子：这张图展示了一只可爱的卡通棕色小熊，它正坐着微笑，有着浅色的肚子和粉色的脚掌。\\n\\n再简单点：这是一只可爱的卡通棕色小熊，正坐着冲人微笑。\\n\\n让我们结合水印信息，虽然用户没问，但图里有“图精灵 616PIC”。不过介绍内容为主。\\n\\n最终句子：这张图展示了一只可爱的卡通棕色小熊，它正坐着微笑，有着粉色的耳朵和脚掌。\\n\\n再修饰一下：这是一张可爱的卡通插画，画着一只坐着的棕色小熊，它有着粉色的耳朵和脚底，表情十分友善。\\n\\n好吧，最自然的一句话：这张图里有一只可爱的卡通棕色小熊，它正坐着微笑，有着浅棕色的肚子和粉色的脚掌。\\n\\n再试一个：这是一只坐着的卡通小熊，全身棕色，有着可爱的粉色脚掌和耳朵，正对着镜头微笑。\\n\\n好，就这个。\\n\\n**修正**：用户问“这张图里有什么？”，然后要求“一句话介绍一下”。\\n\\n回答结构：\\n1. 直接回答有什么（卡通小熊）。\\n2. 一句话描述。\\n\\n“这张图里有一只可爱的卡通棕色小熊。”\\n“这是一只坐着的卡通小熊，有着棕色的皮毛、浅色的肚子以及粉色的耳朵和脚掌，正微笑着看向前方。”\\n\\n合并：这张图里有一只可爱的卡通棕色小熊，它正坐着微笑，有着浅色的肚子和粉色的耳朵及脚掌。\\n\\n再仔细看图，小熊的鼻子是深棕色的，眼睛黑黑的。\\n\\n最终定稿：这张图展示了一只可爱的卡通棕色小熊，它正坐着微笑，有着深棕色的鼻子、粉色的耳朵和脚掌，以及浅色的肚子。\\n\\n简单点好：这是一只可爱的卡通小熊，呈坐姿，全身棕色，有着粉色的耳朵和脚底，正微笑着。\\n\\n好吧，就这样。\\n\\n**最终输出**：这张图里有一只可爱的卡通棕色小熊，它正坐着微笑，有着浅色的肚子和粉色的脚掌。\\n或者：这是一张卡通插画，画着一只坐着的棕色小熊，它有着粉色的耳朵和脚底，表情十分可爱。\\n\\n我觉得“这是一只可爱的卡通棕色小熊，正坐着微笑，有着粉色的耳朵和脚掌。”最稳妥。\\n\\n再补充一下水印？不需要，除非用户特别问。\\n\\n好吧，就介绍主体。\\n\\n**最终答案**：这张图里有一只可爱的卡通棕色小熊，它正坐着微笑，有着浅色的肚子和粉色的耳朵及脚掌。', 'type': 'thinking'}, {'text': '这张图里有一只可爱的卡通棕色小熊，它正坐着微笑，有着浅色的肚子、深色的鼻子以及粉色的耳朵和脚掌。\\n\\n简单来说，这是一张**可爱的卡通小熊插画**，小熊呈坐姿，表情友善呆萌。', 'type': 'text'}] additional_kwargs={} response_metadata={'id': 'msg_fb41298f-9848-4c84-8f42-2b5d36cdee46', 'container': None, 'model': 'qwen3.7-flash', 'stop_details': None, 'stop_reason': 'end_turn', 'stop_sequence': None, 'usage': {'cache_creation': None, 'cache_creation_input_tokens': 0, 'cache_read_input_tokens': 0, 'inference_geo': None, 'input_tokens': 259, 'output_tokens': 919, 'output_tokens_details': None, 'server_tool_use': None, 'service_tier': None, 'prompt_tokens_details': {'cached_tokens': 0}}, 'model_name': 'qwen3.7-flash', 'model_provider': 'anthropic'} id='lc_run--019fd64a-e65a-76e1-a4ba-82c0274cce6f-0' tool_calls=[] invalid_tool_calls=[] usage_metadata={'input_tokens': 259, 'output_tokens': 919, 'total_tokens': 1178, 'input_token_details': {'cache_read': 0, 'cache_creation': 0}}

输入格式化：openai兼容

```
In [18]: from langchain.chat_models import init_chat_model
import os
from dotenv import load_dotenv
import base64
from langchain_core.messages import HumanMessage

load_dotenv()

model = init_chat_model(
    model="openai:qwen3.7-flash",
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    base_url=os.getenv("DASHSCOPE_BASE_URL")
)

def encode_image(img_path, img_type="jpeg"):
```

```

"""请将本地图片转换成 Base64 编码的 Data URI 字符串，方便在文本中嵌入图片数据"""
with open(img_path, "rb") as img_file:
    return f"data:image/{img_type};base64,{base64.b64encode(img_file.read()).decode('utf-8')}"
    # return base64.b64encode(img_file.read()).decode('utf-8')

# 图像路径
img_path = "img_test.jpg"

# 获取图像base64编码字符串
base64_image = encode_image(img_path)

response = model.invoke(
    [ # OpenAI / DashScope / SiliconFlow / DeepSeek :
      HumanMessage(
        content=[
          {"type": 'text', 'text': '这张图里有什么？一句话说明一下'},
          {
            'type': 'image_url',
            'image_url': base64_image,
          }
        ]
      )
    ]
)
# 下面 Anthropic Claude 专属：但是qwen模型好像不行！！
# HumanMessage(
#     content_blocks=[
#         {'type': 'text', 'text': '这张图里有什么？'},
#         {
#             'type': 'image',
#             'base64': base64_image,
#             'mime_type': 'image/jpeg',
#         }
#     ]
# )

print(response)

```

content='这张图里是一只可爱的卡通棕色小熊，它正坐着，有着浅色的肚子、粉红色的脸颊和脚掌，看起来非常呆萌。' additional_kwargs={'refusal': None} response_metadata={'token_usage': {'completion_tokens': 575, 'prompt_tokens': 260, 'total_tokens': 835, 'completion_tokens_details': {'accepted_prediction_tokens': None, 'audio_tokens': None, 'reasoning_tokens': 541, 'rejected_prediction_tokens': None, 'text_tokens': 34}, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0, 'image_tokens': 242, 'text_tokens': 18}}, 'model_provider': 'openai', 'model_name': 'qwen3.7-flash', 'system_fingerprint': None, 'id': 'chatcmpl-280127cb-9265-9f71-83a3-1a71e8b6136d', 'finish_reason': 'stop', 'logprobs': None} id='lc_run--019fd659-29aa-7f02-8aba-d816ab35888f-0' tool_calls=[] invalid_tool_calls=[] usage_metadata={'input_tokens': 260, 'output_tokens': 575, 'total_tokens': 835, 'input_token_details': {'cache_read': 0}, 'output_token_details': {'reasoning': 541}}

输出格式化：deepseek

以deepseek官网的deepseek-v4-flash为例

init_chat_model里增加extra_body={"thinking":{"type":"enabled"}},
输出AIMessage中会有additonal_kwargs,reasoning_content字段

不同的模型输出格式不同，
conten_blocks提供了统一输出格式

```
In [1]: from langchain.chat_models import init_chat_model
import os
from dotenv import load_dotenv

load_dotenv()

model = init_chat_model(
    model="deepseek:deepseek-v4-flash",
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    # base_url=os.getenv("DEEPSEEK_BASE_URL"),
    extra_body={"thinking":{"type":"enabled"}}
)

response = model.invoke("你好，一句话回答")
print(response)
```

```
content='你好！请问有什么我可以一句话帮你解答的吗？' additional_kwargs={'refusal': None,
'reasoning_content': '我们需要理解用户请求。用户说：“你好，一句话回答”。意思是用户希望我们一
句话回答。但用户没有给出具体问题。可能用户只是打招呼并要求一句话回答。我们需要用一句话回
应。可能应该问候并询问有什么需要帮助。保持一句话。注意不要多解释。所以回答：“你好！请问有
什么我可以一句话帮你解答的吗？”这样符合。确保是中文。'} response_metadata={'token_usage':
{'completion_tokens': 88, 'prompt_tokens': 87, 'total_tokens': 175, 'completion_tokens_details':
{'accepted_prediction_tokens': None, 'audio_tokens': None, 'reasoning_tokens': 77,
'rejected_prediction_tokens': None}, 'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 0}, 'prompt_cache_hit_tokens': 0, 'prompt_cache_miss_tokens': 87},
'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402', 'id': '87efaabb-7e78-4d43-a050-
9947dc606944', 'finish_reason': 'stop', 'logprobs': None} id='lc_run--019ff9cb-8d79-7132-a449-
78358aa12af2-0' tool_calls=[] invalid_tool_calls=[] usage_metadata={'input_tokens': 87,
'output_tokens': 88, 'total_tokens': 175, 'input_token_details': {'cache_read': 0},
'output_token_details': {'reasoning': 77}}
```

```
In [2]: print(response.content)
```

你好！请问有什么我可以一句话帮你解答的吗？

```
In [3]: print(response.content_blocks)
```

```
[{'type': 'reasoning', 'reasoning': '我们需要理解用户请求。用户说：“你好，一句话回答”。意思是用户希望我们一句话回答。但用户没有给出具体问题。可能用户只是打招呼并要求一句话回答。我们需要用一句话回应。可能应该问候并询问有什么需要帮助。保持一句话。注意不要多解释。所以回答：“你好！请问有什么我可以一句话帮你解答的吗？”这样符合。确保是中文。'}, {'type': 'text', 'text': '你好！请问有什么我可以一句话帮你解答的吗？'}]
```